LAB MANUAL

OOP

QUESTION # 1:

Create a `BankAccount` class that represents a bank account. It should have attributes for account number, account holder name, and balance. Include methods to deposit, withdraw, and display balance. Ensure withdrawals cannot exceed the available balance.

CODE:

```
class BankAccount:
    def __init__(self, account_number, account_holder_name, balance=0):
        """Initialize a bank account with account number, holder name, and
balance"""
        self.account_number = account_number
        self.account_holder_name = account_holder_name
        self.balance = balance

    def deposit(self, amount):
        """Deposit money into the account"""
        if amount > 0:
            self.balance += amount
            print(f"${amount} deposited successfully!")
            print(f"New balance: ${self.balance}")
        else:
            print("Deposit amount must be positive!")

    def withdraw(self, amount):
        """Withdraw money from the account"""
        if amount <= 0:
            print("Withdrawal amount must be positive!")
        elif amount > self.balance:
            print("Insufficient balance! Withdrawal failed.")
            print(f"Available balance: ${self.balance}")
        else:
            self.balance -= amount
            print(f"${amount} withdrawn successfully!")
            print(f"Remaining balance: ${self.balance}")

    def display_balance(self):
        """Display current account balance"""
        print(f"\n--- Account Details ---")
        print(f"Account Number: {self.account_number}")
        print(f"Account Holder: {self.account_holder_name}")
        print(f"Current Balance: ${self.balance}")
        print(f"-----------------------\n")


# Example Usage
if __name__ == "__main__":
    # Create a new bank account
    account1 = BankAccount("ACC123456", "John Doe", 1000)

    # Display initial balance
    account1.display_balance()

    # Deposit money
    account1.deposit(500)

    # Withdraw money (valid)
    account1.withdraw(300)

    # Try to withdraw more than available balance
    account1.withdraw(2000)

    # Display final balance
```

```
    account1.display_balance()

    # Create another account with zero initial balance
    account2 = BankAccount("ACC789012", "Jane Smith")
    account2.display_balance()
    account2.deposit(250)
```

OUTPUT:

```
--- Account Details ---
Account Number: ACC123456
Account Holder: John Doe
Current Balance: $1000
------------------------
$500 deposited successfully!
New balance: $1500
$300 withdrawn successfully!
Remaining balance: $1200
Insufficient balance! Withdrawal failed.
Available balance: $1200
--- Account Details ---
Account Number: ACC123456
Account Holder: John Doe
Current Balance: $1200
------------------------
--- Account Details ---
Account Number: ACC789012
Account Holder: Jane Smith
Current Balance: $0
------------------------
$250 deposited successfully!
New balance: $250
```

QUESTION # 2:

Design a `Book` class with attributes like title, author, ISBN, and availability status. Create a `Library` class that can add books, search for books by title/author, check out books, and return books. Track which books are currently available.

CODE:

```
class Book:
    def __init__(self, title, author, isbn):
        """Initialize a book with title, author, ISBN, and availability status"""
        self.title = title
        self.author = author
        self.isbn = isbn
        self.is_available = True  # Book is available by default

    def display_info(self):
        """Display book information"""
        status = "Available" if self.is_available else "Checked Out"
        print(f"Title: {self.title}")
        print(f"Author: {self.author}")
        print(f"ISBN: {self.isbn}")
        print(f"Status: {status}")
        print("-" * 40)


class Library:
    def __init__(self, library_name):
        """Initialize a library with a name and empty book list"""
```

```
        self.library_name = library_name
        self.books = []

    def add_book(self, book):
        """Add a book to the library"""
        self.books.append(book)
        print(f"Book '{book.title}' added to the library successfully!")

    def search_by_title(self, title):
        """Search for books by title"""
        print(f"\nSearching for books with title: '{title}'")
        print("=" * 40)
        found = False
        for book in self.books:
            if title.lower() in book.title.lower():
                book.display_info()
                found = True
        if not found:
            print("No books found with that title.")

    def search_by_author(self, author):
        """Search for books by author"""
        print(f"\nSearching for books by author: '{author}'")
        print("=" * 40)
        found = False
        for book in self.books:
            if author.lower() in book.author.lower():
                book.display_info()
                found = True
        if not found:
            print("No books found by that author.")

    def check_out_book(self, isbn):
        """Check out a book using its ISBN"""
        for book in self.books:
            if book.isbn == isbn:
                if book.is_available:
                    book.is_available = False
                    print(f"\nBook '{book.title}' checked out successfully!")
                    return
                else:
                    print(f"\nSorry, '{book.title}' is already checked out.")
                    return
        print(f"\nBook with ISBN '{isbn}' not found in the library.")

    def return_book(self, isbn):
        """Return a book using its ISBN"""
        for book in self.books:
            if book.isbn == isbn:
                if not book.is_available:
                    book.is_available = True
                    print(f"\nBook '{book.title}' returned successfully!")
                    return
                else:
                    print(f"\nBook '{book.title}' was not checked out.")
                    return
        print(f"\nBook with ISBN '{isbn}' not found in the library.")

    def display_available_books(self):
        """Display all available books"""
        print(f"\n{'='*40}")
        print(f"Available Books in {self.library_name}")
        print(f"{'='*40}")
        available_books = [book for book in self.books if book.is_available]
        if available_books:
            for book in available_books:
                book.display_info()
        else:
            print("No books are currently available.")

    def display_all_books(self):
        """Display all books in the library"""
        print(f"\n{'='*40}")
        print(f"All Books in {self.library_name}")
        print(f"{'='*40}")
```

```
        if self.books:
            for book in self.books:
                book.display_info()
        else:
            print("The library has no books.")


# Example Usage
if __name__ == "__main__":
    # Create a library
    my_library = Library("City Public Library")

    # Create books
    book1 = Book("Python Programming", "John Smith", "ISBN001")
    book2 = Book("Data Science Basics", "Jane Doe", "ISBN002")
    book3 = Book("Machine Learning", "John Smith", "ISBN003")
    book4 = Book("Artificial Intelligence", "Alice Johnson", "ISBN004")

    # Add books to library
    my_library.add_book(book1)
    my_library.add_book(book2)
    my_library.add_book(book3)
    my_library.add_book(book4)

    # Display all books
    my_library.display_all_books()

    # Search by title
    my_library.search_by_title("Python")

    # Search by author
    my_library.search_by_author("John Smith")

    # Check out a book
    my_library.check_out_book("ISBN001")

    # Try to check out the same book again
    my_library.check_out_book("ISBN001")

    # Display available books
    my_library.display_available_books()

    # Return the book
    my_library.return_book("ISBN001")

    # Display available books again
    my_library.display_available_books()
```

OUTPUT:

```
Book 'Python Programming' added to the library successfully!
Book 'Data Science Basics' added to the library successfully!
Book 'Machine Learning' added to the library successfully!
Book 'Artificial Intelligence' added to the library successfully!

========================================
All Books in City Public Library
========================================
Title: Python Programming
Author: John Smith
ISBN: ISBN001
Status: Available
----------------------------------------
Title: Data Science Basics
Author: Jane Doe
ISBN: ISBN002
```

```
Status: Available
----------------------------------------
Title: Machine Learning
Author: John Smith
ISBN: ISBN003
Status: Available
----------------------------------------
Title: Artificial Intelligence
Author: Alice Johnson
ISBN: ISBN004
Status: Available
----------------------------------------

Searching for books with title: 'Python'
========================================
Title: Python Programming
Author: John Smith
ISBN: ISBN001
Status: Available
----------------------------------------

Searching for books by author: 'John Smith'
========================================
Title: Python Programming
Author: John Smith
ISBN: ISBN001
Status: Available
----------------------------------------
Title: Machine Learning
Author: John Smith
ISBN: ISBN003
Status: Available
----------------------------------------

Book 'Python Programming' checked out successfully!

Sorry, 'Python Programming' is already checked out.

========================================
Available Books in City Public Library
========================================
Title: Data Science Basics
Author: Jane Doe
ISBN: ISBN002
Status: Available
----------------------------------------
Title: Machine Learning
Author: John Smith
ISBN: ISBN003
Status: Available
----------------------------------------
Title: Artificial Intelligence
Author: Alice Johnson
ISBN: ISBN004
Status: Available
----------------------------------------
```

```
Book 'Python Programming' returned successfully!

========================================
Available Books in City Public Library
========================================
Title: Python Programming
Author: John Smith
ISBN: ISBN001
Status: Available
----------------------------------------
Title: Data Science Basics
Author: Jane Doe
ISBN: ISBN002
Status: Available
----------------------------------------
Title: Machine Learning
Author: John Smith
ISBN: ISBN003
Status: Available
----------------------------------------
Title: Artificial Intelligence
Author: Alice Johnson
ISBN: ISBN004
Status: Available
----------------------------------------
```

QUESTION NO 3:

Create a `Student` class with attributes for student ID, name, and a list of grades. Implement methods to add grades, calculate average grade, and determine the letter grade (A, B, C, etc.). Add a method to display student information with their performance summary.

CODE:

```python
class Student:
    def __init__(self, student_id, name):
        """Initialize a student with ID, name, and empty grades list"""
        self.student_id = student_id
        self.name = name
        self.grades = []

    def add_grade(self, grade):
        """Add a grade to the student's grades list"""
        if 0 <= grade <= 100:
            self.grades.append(grade)
            print(f"Grade {grade} added successfully for {self.name}!")
        else:
            print("Invalid grade! Grade must be between 0 and 100.")

    def calculate_average(self):
        """Calculate and return the average grade"""
        if len(self.grades) == 0:
            return 0
        total = sum(self.grades)
        average = total / len(self.grades)
        return round(average, 2)

    def get_letter_grade(self):
        """Determine letter grade based on average"""
        average = self.calculate_average()
```

```
        if average >= 90:
            return 'A'
        elif average >= 80:
            return 'B'
        elif average >= 70:
            return 'C'
        elif average >= 60:
            return 'D'
        else:
            return 'F'

    def display_student_info(self):
        """Display complete student information with performance summary"""
        print("\n" + "="*50)
        print("STUDENT INFORMATION & PERFORMANCE SUMMARY")
        print("="*50)
        print(f"Student ID: {self.student_id}")
        print(f"Student Name: {self.name}")
        print(f"Number of Grades: {len(self.grades)}")

        if len(self.grades) > 0:
            print(f"Grades: {self.grades}")
            print(f"Average Grade: {self.calculate_average()}")
            print(f"Letter Grade: {self.get_letter_grade()}")

            # Performance summary
            letter = self.get_letter_grade()
            if letter == 'A':
                performance = "Excellent"
            elif letter == 'B':
                performance = "Good"
            elif letter == 'C':
                performance = "Satisfactory"
            elif letter == 'D':
                performance = "Needs Improvement"
            else:
                performance = "Failing"

            print(f"Performance: {performance}")
        else:
            print("No grades recorded yet.")

        print("="*50 + "\n")


# Example Usage
if __name__ == "__main__":
    # Create students
    student1 = Student("S001", "Alice Johnson")
    student2 = Student("S002", "Bob Smith")
    student3 = Student("S003", "Charlie Brown")

    # Add grades for student1
    print("\n--- Adding grades for Alice ---")
    student1.add_grade(95)
    student1.add_grade(88)
    student1.add_grade(92)
    student1.add_grade(90)

    # Display student1 info
    student1.display_student_info()

    # Add grades for student2
    print("--- Adding grades for Bob ---")
    student2.add_grade(75)
    student2.add_grade(78)
    student2.add_grade(72)
    student2.add_grade(80)
    student2.add_grade(77)

    # Display student2 info
    student2.display_student_info()

    # Add grades for student3
```

```
print("--- Adding grades for Charlie ---")
student3.add_grade(55)
student3.add_grade(60)
student3.add_grade(58)

# Try to add invalid grade
student3.add_grade(105)  # Invalid grade

# Display student3 info
student3.display_student_info()

# Create a student with no grades
student4 = Student("S004", "Diana Prince")
student4.display_student_info()
```

OUTPUT:

```
--- Adding grades for Alice ---
Grade 95 added successfully for Alice Johnson!
Grade 88 added successfully for Alice Johnson!
Grade 92 added successfully for Alice Johnson!
Grade 90 added successfully for Alice Johnson!

==================================================
STUDENT INFORMATION & PERFORMANCE SUMMARY
==================================================
Student ID: S001
Student Name: Alice Johnson
Number of Grades: 4
Grades: [95, 88, 92, 90]
Average Grade: 91.25
Letter Grade: A
Performance: Excellent
==================================================

--- Adding grades for Bob ---
Grade 75 added successfully for Bob Smith!
Grade 78 added successfully for Bob Smith!
Grade 72 added successfully for Bob Smith!
Grade 80 added successfully for Bob Smith!
Grade 77 added successfully for Bob Smith!

==================================================
STUDENT INFORMATION & PERFORMANCE SUMMARY
==================================================
Student ID: S002
Student Name: Bob Smith
Number of Grades: 5
Grades: [75, 78, 72, 80, 77]
Average Grade: 76.4
Letter Grade: C
Performance: Satisfactory
==================================================

--- Adding grades for Charlie ---
Grade 55 added successfully for Charlie Brown!
Grade 60 added successfully for Charlie Brown!
Grade 58 added successfully for Charlie Brown!
Invalid grade! Grade must be between 0 and 100.
```

```
==================================================
STUDENT INFORMATION & PERFORMANCE SUMMARY
==================================================
Student ID: S003
Student Name: Charlie Brown
Number of Grades: 3
Grades: [55, 60, 58]
Average Grade: 57.67
Letter Grade: F
Performance: Failing
==================================================


==================================================
STUDENT INFORMATION & PERFORMANCE SUMMARY
==================================================
Student ID: S004
Student Name: Diana Prince
Number of Grades: 0
No grades recorded yet.
==================================================
```

QUESTION NO 4:

Design a `MenuItem` class for food items (name, price, category) and an `Order` class that can add multiple items, calculate total cost, apply discounts, and generate receipts. Include tax calculation functionality

---

CODE:

```python
class MenuItem:
    def __init__(self, name, price, category):
        """Initialize a menu item with name, price, and category"""
        self.name = name
        self.price = price
        self.category = category

    def display_item(self):
        """Display menu item information"""
        print(f"{self.name} - ${self.price:.2f} ({self.category})")


class Order:
    def __init__(self, order_id, tax_rate=0.08):
        """Initialize an order with order ID and tax rate"""
        self.order_id = order_id
        self.items = []
        self.tax_rate = tax_rate
        self.discount_percentage = 0

    def add_item(self, menu_item):
        """Add a menu item to the order"""
        self.items.append(menu_item)
        print(f"'{menu_item.name}' added to order!")

    def calculate_subtotal(self):
        """Calculate subtotal (sum of all item prices)"""
        subtotal = sum(item.price for item in self.items)
        return subtotal

    def apply_discount(self, discount_percentage):
        """Apply discount percentage to the order"""
```

```
        if 0 <= discount_percentage <= 100:
            self.discount_percentage = discount_percentage
            print(f"Discount of {discount_percentage}% applied!")
        else:
            print("Invalid discount! Must be between 0 and 100.")

    def calculate_discount_amount(self):
        """Calculate the discount amount"""
        subtotal = self.calculate_subtotal()
        discount_amount = subtotal * (self.discount_percentage / 100)
        return discount_amount

    def calculate_tax(self):
        """Calculate tax on the discounted amount"""
        subtotal = self.calculate_subtotal()
        discount_amount = self.calculate_discount_amount()
        amount_after_discount = subtotal - discount_amount
        tax = amount_after_discount * self.tax_rate
        return tax

    def calculate_total(self):
        """Calculate total cost including tax and discount"""
        subtotal = self.calculate_subtotal()
        discount_amount = self.calculate_discount_amount()
        tax = self.calculate_tax()
        total = subtotal - discount_amount + tax
        return total

    def generate_receipt(self):
        """Generate and display detailed receipt"""
        print("\n" + "="*50)
        print("           RESTAURANT RECEIPT")
        print("="*50)
        print(f"Order ID: {self.order_id}")
        print("-"*50)
        print("ITEMS ORDERED:")
        print("-"*50)

        if len(self.items) == 0:
            print("No items in order.")
        else:
            for i, item in enumerate(self.items, 1):
                print(f"{i}. {item.name:30s} ${item.price:6.2f}")
                print(f"   Category: {item.category}")

        print("-"*50)

        if len(self.items) > 0:
            subtotal = self.calculate_subtotal()
            discount_amount = self.calculate_discount_amount()
            tax = self.calculate_tax()
            total = self.calculate_total()

            print(f"Subtotal:                        ${subtotal:6.2f}")

            if self.discount_percentage > 0:
                print(f"Discount ({self.discount_percentage}%):                -
${discount_amount:6.2f}")
                print(f"Amount after discount:         ${subtotal -
discount_amount:6.2f}")

            print(f"Tax ({self.tax_rate*100}%):                     ${tax:6.2f}")
            print("-"*50)
            print(f"TOTAL:                           ${total:6.2f}")

        print("="*50)
        print("       Thank you for your order!")
        print("="*50 + "\n")


# Example Usage
if __name__ == "__main__":
    # Create menu items
    item1 = MenuItem("Burger", 8.99, "Main Course")
    item2 = MenuItem("French Fries", 3.99, "Side Dish")
```

```
item3 = MenuItem("Cola", 2.50, "Beverage")
item4 = MenuItem("Pizza", 12.99, "Main Course")
item5 = MenuItem("Ice Cream", 4.50, "Dessert")
item6 = MenuItem("Salad", 6.99, "Appetizer")

# Create first order
print("\n--- ORDER 1 ---")
order1 = Order("ORD001", tax_rate=0.08)

# Add items to order1
order1.add_item(item1)
order1.add_item(item2)
order1.add_item(item3)

# Generate receipt without discount
order1.generate_receipt()

# Create second order with discount
print("\n--- ORDER 2 ---")
order2 = Order("ORD002", tax_rate=0.10)

# Add items to order2
order2.add_item(item4)
order2.add_item(item5)
order2.add_item(item6)
order2.add_item(item3)

# Apply discount
order2.apply_discount(15)

# Generate receipt with discount
order2.generate_receipt()

# Create third order - empty order
print("\n--- ORDER 3 ---")
order3 = Order("ORD003")
order3.generate_receipt()

# Create fourth order with large discount
print("\n--- ORDER 4 ---")
order4 = Order("ORD004", tax_rate=0.07)
order4.add_item(item1)
order4.add_item(item4)
order4.add_item(item5)
order4.apply_discount(20)
order4.generate_receipt()
```

OUTPUT:

```
--- ORDER 1 ---
'Burger' added to order!
'French Fries' added to order!
'Cola' added to order!

================================================
           RESTAURANT RECEIPT
================================================
Order ID: ORD001
------------------------------------------------
ITEMS ORDERED:
------------------------------------------------
1. Burger                       $  8.99
   Category: Main Course
2. French Fries                 $  3.99
   Category: Side Dish
3. Cola                         $  2.50
   Category: Beverage
```

```
--------------------------------------------------
Subtotal:                        $ 15.48
Tax (8.0%):                      $  1.24
--------------------------------------------------
TOTAL:                           $ 16.72
==================================================
       Thank you for your order!
==================================================


--- ORDER 2 ---
'Pizza' added to order!
'Ice Cream' added to order!
'Salad' added to order!
'Cola' added to order!
Discount of 15% applied!

==================================================
           RESTAURANT RECEIPT
==================================================
Order ID: ORD002
--------------------------------------------------
ITEMS ORDERED:
--------------------------------------------------
1. Pizza                           $ 12.99
   Category: Main Course
2. Ice Cream                       $  4.50
   Category: Dessert
3. Salad                           $  6.99
   Category: Appetizer
4. Cola                            $  2.50
   Category: Beverage
--------------------------------------------------
Subtotal:                        $ 26.98
Discount (15%):                 -$  4.05
Amount after discount:           $ 22.93
Tax (10.0%):                      $  2.29
--------------------------------------------------
TOTAL:                           $ 25.22
==================================================
       Thank you for your order!
==================================================


--- ORDER 3 ---

==================================================
           RESTAURANT RECEIPT
==================================================
Order ID: ORD003
--------------------------------------------------
ITEMS ORDERED:
--------------------------------------------------
No items in order.
--------------------------------------------------
==================================================
```

```
       Thank you for your order!
================================================


--- ORDER 4 ---
'Burger' added to order!
'Pizza' added to order!
'Ice Cream' added to order!
Discount of 20% applied!

================================================
           RESTAURANT RECEIPT
================================================
Order ID: ORD004
------------------------------------------------
ITEMS ORDERED:
------------------------------------------------
1. Burger                        $  8.99
   Category: Main Course
2. Pizza                         $ 12.99
   Category: Main Course
3. Ice Cream                     $  4.50
   Category: Dessert
------------------------------------------------
Subtotal:                       $ 26.48
Discount (20%):                -$  5.30
Amount after discount:          $ 21.18
Tax (7.0%):                     $  1.48
------------------------------------------------
TOTAL:                          $ 22.66
================================================
       Thank you for your order!
================================================
```

QUESTION NO 5:

Create classes for `Vehicle` (base class) and derived classes `Car`, `Motorcycle`, and `Truck`. Each vehicle should have attributes like license plate, model, rental price per day, and availability. Implement a system to rent vehicles and calculate rental costs.

CODE

```python
class vehicle:
    def __init__(self, license_plate, model, rental_price_per_day):
        """Initialize base vehicle with common attributes"""
        self.license_plate = license_plate
        self.model = model
        self.rental_price_per_day = rental_price_per_day
        self.is_available = True

    def display_info(self):
        """Display vehicle information"""
        status = "Available" if self.is_available else "Rented"
        print(f"License Plate: {self.license_plate}")
        print(f"Model: {self.model}")
        print(f"Rental Price: ${self.rental_price_per_day}/day")
        print(f"Status: {status}")
```

```
    def rent_vehicle(self, days):
        """Rent the vehicle for specified days"""
        if self.is_available:
            self.is_available = False
            total_cost = self.rental_price_per_day * days
            print(f"\n{self.model} rented successfully!")
            print(f"Rental Period: {days} days")
            print(f"Total Cost: ${total_cost:.2f}")
            return total_cost
        else:
            print(f"\nSorry, {self.model} is already rented.")
            return 0

    def return_vehicle(self):
        """Return the rented vehicle"""
        if not self.is_available:
            self.is_available = True
            print(f"\n{self.model} returned successfully!")
        else:
            print(f"\n{self.model} was not rented.")


class Car(Vehicle):
    def __init__(self, license_plate, model, rental_price_per_day, num_seats):
        """Initialize Car with additional attribute"""
        super().__init__(license_plate, model, rental_price_per_day)
        self.num_seats = num_seats
        self.vehicle_type = "Car"

    def display_info(self):
        """Display car information with additional details"""
        print(f"\nVehicle Type: {self.vehicle_type}")
        super().display_info()
        print(f"Number of Seats: {self.num_seats}")
        print("-" * 40)


class Motorcycle(Vehicle):
    def __init__(self, license_plate, model, rental_price_per_day,
engine_capacity):
        """Initialize Motorcycle with additional attribute"""
        super().__init__(license_plate, model, rental_price_per_day)
        self.engine_capacity = engine_capacity
        self.vehicle_type = "Motorcycle"

    def display_info(self):
        """Display motorcycle information with additional details"""
        print(f"\nVehicle Type: {self.vehicle_type}")
        super().display_info()
        print(f"Engine Capacity: {self.engine_capacity}cc")
        print("-" * 40)


class Truck(Vehicle):
    def __init__(self, license_plate, model, rental_price_per_day, cargo_capacity):
        """Initialize Truck with additional attribute"""
        super().__init__(license_plate, model, rental_price_per_day)
        self.cargo_capacity = cargo_capacity
        self.vehicle_type = "Truck"

    def display_info(self):
        """Display truck information with additional details"""
        print(f"\nVehicle Type: {self.vehicle_type}")
        super().display_info()
        print(f"Cargo Capacity: {self.cargo_capacity} tons")
        print("-" * 40)


class RentalSystem:
    def __init__(self):
        """Initialize rental system with empty vehicle list"""
        self.vehicles = []

    def add_vehicle(self, vehicle):
        """Add a vehicle to the rental system"""
```

```
        self.vehicles.append(vehicle)
        print(f"{vehicle.model} added to rental system!")

    def display_all_vehicles(self):
        """Display all vehicles in the system"""
        print("\n" + "="*50)
        print("ALL VEHICLES IN RENTAL SYSTEM")
        print("="*50)
        for vehicle in self.vehicles:
            vehicle.display_info()

    def display_available_vehicles(self):
        """Display only available vehicles"""
        print("\n" + "="*50)
        print("AVAILABLE VEHICLES")
        print("="*50)
        available = [v for v in self.vehicles if v.is_available]
        if available:
            for vehicle in available:
                vehicle.display_info()
        else:
            print("No vehicles available at the moment.")


# Example Usage
if __name__ == "__main__":
    # Create rental system
    rental_system = RentalSystem()

    # Create vehicles
    car1 = Car("ABC123", "Toyota Camry", 50, 5)
    car2 = Car("XYZ789", "Honda Civic", 45, 5)
    motorcycle1 = Motorcycle("MOTO001", "Yamaha R15", 25, 150)
    motorcycle2 = Motorcycle("MOTO002", "Honda CBR", 30, 250)
    truck1 = Truck("TRK001", "Ford F-150", 80, 2.5)
    truck2 = Truck("TRK002", "Chevrolet Silverado", 85, 3.0)

    # Add vehicles to rental system
    print("\n--- Adding Vehicles ---")
    rental_system.add_vehicle(car1)
    rental_system.add_vehicle(car2)
    rental_system.add_vehicle(motorcycle1)
    rental_system.add_vehicle(motorcycle2)
    rental_system.add_vehicle(truck1)
    rental_system.add_vehicle(truck2)

    # Display all vehicles
    rental_system.display_all_vehicles()

    # Rent some vehicles
    print("\n" + "="*50)
    print("RENTING VEHICLES")
    print("="*50)
    car1.rent_vehicle(3)
    motorcycle1.rent_vehicle(5)
    truck1.rent_vehicle(2)

    # Try to rent already rented vehicle
    car1.rent_vehicle(2)

    # Display available vehicles
    rental_system.display_available_vehicles()

    # Return a vehicle
    print("\n" + "="*50)
    print("RETURNING VEHICLES")
    print("="*50)
    car1.return_vehicle()

    # Display available vehicles again
    rental_system.display_available_vehicles()

    # Calculate rental for different periods
    print("\n" + "="*50)
    print("ADDITIONAL RENTALS")
```

```
print("="*50)
car2.rent_vehicle(7)
truck2.rent_vehicle(4)
```

OUTPUT:

```
--- Adding Vehicles ---
Toyota Camry added to rental system!
Honda Civic added to rental system!
Yamaha R15 added to rental system!
Honda CBR added to rental system!
Ford F-150 added to rental system!
Chevrolet Silverado added to rental system!

==================================================
ALL VEHICLES IN RENTAL SYSTEM
==================================================

Vehicle Type: Car
License Plate: ABC123
Model: Toyota Camry
Rental Price: $50/day
Status: Available
Number of Seats: 5
----------------------------------------

Vehicle Type: Car
License Plate: XYZ789
Model: Honda Civic
Rental Price: $45/day
Status: Available
Number of Seats: 5
----------------------------------------

Vehicle Type: Motorcycle
License Plate: MOTO001
Model: Yamaha R15
Rental Price: $25/day
Status: Available
Engine Capacity: 150cc
----------------------------------------

Vehicle Type: Motorcycle
License Plate: MOTO002
Model: Honda CBR
Rental Price: $30/day
Status: Available
Engine Capacity: 250cc
----------------------------------------

Vehicle Type: Truck
License Plate: TRK001
Model: Ford F-150
Rental Price: $80/day
Status: Available
Cargo Capacity: 2.5 tons
```

```
----------------------------------------

Vehicle Type: Truck
License Plate: TRK002
Model: Chevrolet Silverado
Rental Price: $85/day
Status: Available
Cargo Capacity: 3.0 tons
----------------------------------------

================================================
RENTING VEHICLES
================================================

Toyota Camry rented successfully!
Rental Period: 3 days
Total Cost: $150.00

Yamaha R15 rented successfully!
Rental Period: 5 days
Total Cost: $125.00

Ford F-150 rented successfully!
Rental Period: 2 days
Total Cost: $160.00

Sorry, Toyota Camry is already rented.

================================================
AVAILABLE VEHICLES
================================================

Vehicle Type: Car
License Plate: XYZ789
Model: Honda Civic
Rental Price: $45/day
Status: Available
Number of Seats: 5
----------------------------------------

Vehicle Type: Motorcycle
License Plate: MOTO002
Model: Honda CBR
Rental Price: $30/day
Status: Available
Engine Capacity: 250cc
----------------------------------------

Vehicle Type: Truck
License Plate: TRK002
Model: Chevrolet Silverado
Rental Price: $85/day
Status: Available
Cargo Capacity: 3.0 tons
----------------------------------------
```

```
==================================================
RETURNING VEHICLES
==================================================

Toyota Camry returned successfully!

==================================================
AVAILABLE VEHICLES
==================================================

Vehicle Type: Car
License Plate: ABC123
Model: Toyota Camry
Rental Price: $50/day
Status: Available
Number of Seats: 5
----------------------------------------

Vehicle Type: Car
License Plate: XYZ789
Model: Honda Civic
Rental Price: $45/day
Status: Available
Number of Seats: 5
----------------------------------------

Vehicle Type: Motorcycle
License Plate: MOTO002
Model: Honda CBR
Rental Price: $30/day
Status: Available
Engine Capacity: 250cc
----------------------------------------

Vehicle Type: Truck
License Plate: TRK002
Model: Chevrolet Silverado
Rental Price: $85/day
Status: Available
Cargo Capacity: 3.0 tons
----------------------------------------

==================================================
ADDITIONAL RENTALS
==================================================

Honda Civic rented successfully!
Rental Period: 7 days
Total Cost: $315.00

Chevrolet Silverado rented successfully!
Rental Period: 4 days
Total Cost: $340.00
```

QUESTION NO 6:

Design an `Employee` class with attributes for employee ID, name, position, and salary. Create methods to calculate monthly salary, annual salary, and apply raises. Include functionality to track employee details and employment duration.

CODE:

```
from datetime import datetime

class Employee:
    def __init__(self, employee_id, name, position, salary, hire_date):
        """Initialize employee with ID, name, position, salary, and hire date"""
        self.employee_id = employee_id
        self.name = name
        self.position = position
        self.salary = salary  # Annual salary
        self.hire_date = hire_date

    def calculate_monthly_salary(self):
        """Calculate monthly salary"""
        monthly_salary = self.salary / 12
        return round(monthly_salary, 2)

    def calculate_annual_salary(self):
        """Return annual salary"""
        return self.salary

    def apply_raise(self, percentage):
        """Apply a percentage raise to the salary"""
        if percentage > 0:
            raise_amount = self.salary * (percentage / 100)
            old_salary = self.salary
            self.salary += raise_amount
            print(f"\nRaise applied to {self.name}!")
            print(f"Raise Percentage: {percentage}%")
            print(f"Old Salary: ${old_salary:.2f}")
            print(f"Raise Amount: ${raise_amount:.2f}")
            print(f"New Salary: ${self.salary:.2f}")
        else:
            print("Raise percentage must be positive!")

    def calculate_employment_duration(self):
        """Calculate employment duration in years and months"""
        hire_date_obj = datetime.strptime(self.hire_date, "%Y-%m-%d")
        current_date = datetime.now()

        total_days = (current_date - hire_date_obj).days
        years = total_days // 365
        remaining_days = total_days % 365
        months = remaining_days // 30

        return years, months

    def display_employee_details(self):
        """Display complete employee information"""
        years, months = self.calculate_employment_duration()

        print("\n" + "="*50)
        print("EMPLOYEE DETAILS")
        print("="*50)
        print(f"Employee ID: {self.employee_id}")
        print(f"Name: {self.name}")
        print(f"Position: {self.position}")
        print(f"Hire Date: {self.hire_date}")
        print(f"Employment Duration: {years} years, {months} months")
        print("-"*50)
        print(f"Annual Salary: ${self.calculate_annual_salary():,.2f}")
        print(f"Monthly Salary: ${self.calculate_monthly_salary():,.2f}")
        print("="*50 + "\n")


class EmployeeManagementSystem:
    def __init__(self):
```

```
        """Initialize employee management system"""
        self.employees = []

    def add_employee(self, employee):
        """Add an employee to the system"""
        self.employees.append(employee)
        print(f"Employee '{employee.name}' added successfully!")

    def display_all_employees(self):
        """Display all employees"""
        print("\n" + "="*50)
        print("ALL EMPLOYEES")
        print("="*50)
        if self.employees:
            for emp in self.employees:
                emp.display_employee_details()
        else:
            print("No employees in the system.")

    def find_employee_by_id(self, employee_id):
        """Find and return employee by ID"""
        for emp in self.employees:
            if emp.employee_id == employee_id:
                return emp
        return None

    def display_total_payroll(self):
        """Calculate and display total annual payroll"""
        total_annual = sum(emp.salary for emp in self.employees)
        total_monthly = sum(emp.calculate_monthly_salary() for emp in
self.employees)

        print("\n" + "="*50)
        print("PAYROLL SUMMARY")
        print("="*50)
        print(f"Total Employees: {len(self.employees)}")
        print(f"Total Annual Payroll: ${total_annual:,.2f}")
        print(f"Total Monthly Payroll: ${total_monthly:,.2f}")
        print("="*50 + "\n")


# Example Usage
if __name__ == "__main__":
    # Create employee management system
    system = EmployeeManagementSystem()

    # Create employees
    print("\n--- Adding Employees ---")
    emp1 = Employee("E001", "John Smith", "Software Engineer", 75000, "2020-03-15")
    emp2 = Employee("E002", "Sarah Johnson", "Project Manager", 85000, "2019-06-
20")
    emp3 = Employee("E003", "Mike Brown", "Data Analyst", 65000, "2021-01-10")
    emp4 = Employee("E004", "Emily Davis", "HR Manager", 70000, "2018-09-05")

    # Add employees to system
    system.add_employee(emp1)
    system.add_employee(emp2)
    system.add_employee(emp3)
    system.add_employee(emp4)

    # Display all employees
    system.display_all_employees()

    # Apply raises
    print("\n--- Applying Raises ---")
    emp1.apply_raise(10)
    emp3.apply_raise(7.5)

    # Display updated employee details
    print("\n--- Updated Employee Details ---")
    emp1.display_employee_details()
    emp3.display_employee_details()

    # Find employee by ID
    print("\n--- Finding Employee ---")
```

```
found_emp = system.find_employee_by_id("E002")
if found_emp:
    print(f"Employee found: {found_emp.name}")
    found_emp.display_employee_details()

# Display total payroll
system.display_total_payroll()

# Calculate specific details for an employee
print("\n--- Specific Calculations ---")
print(f"{emp2.name}'s Monthly Salary: ${emp2.calculate_monthly_salary():,.2f}")
print(f"{emp2.name}'s Annual Salary: ${emp2.calculate_annual_salary():,.2f}")
years, months = emp2.calculate_employment_duration()
print(f"{emp2.name}'s Employment Duration: {years} years, {months} months")
```

OUTPUT

```
--- Adding Employees ---
Employee 'John Smith' added successfully!
Employee 'Sarah Johnson' added successfully!
Employee 'Mike Brown' added successfully!
Employee 'Emily Davis' added successfully!

==================================================
ALL EMPLOYEES
==================================================

==================================================
EMPLOYEE DETAILS
==================================================
Employee ID: E001
Name: John Smith
Position: Software Engineer
Hire Date: 2020-03-15
Employment Duration: 5 years, 6 months
--------------------------------------------------
Annual Salary: $75,000.00
Monthly Salary: $6,250.00
==================================================


==================================================
EMPLOYEE DETAILS
==================================================
Employee ID: E002
Name: Sarah Johnson
Position: Project Manager
Hire Date: 2019-06-20
Employment Duration: 6 years, 3 months
--------------------------------------------------
Annual Salary: $85,000.00
Monthly Salary: $7,083.33
==================================================


==================================================
EMPLOYEE DETAILS
==================================================
Employee ID: E003
```

```
Name: Mike Brown
Position: Data Analyst
Hire Date: 2021-01-10
Employment Duration: 4 years, 9 months
--------------------------------------------------
Annual Salary: $65,000.00
Monthly Salary: $5,416.67
==================================================


==================================================
EMPLOYEE DETAILS
==================================================
Employee ID: E004
Name: Emily Davis
Position: HR Manager
Hire Date: 2018-09-05
Employment Duration: 7 years, 1 months
--------------------------------------------------
Annual Salary: $70,000.00
Monthly Salary: $5,833.33
==================================================


--- Applying Raises ---

Raise applied to John Smith!
Raise Percentage: 10%
Old Salary: $75000.00
Raise Amount: $7500.00
New Salary: $82500.00

Raise applied to Mike Brown!
Raise Percentage: 7.5%
Old Salary: $65000.00
Raise Amount: $4875.00
New Salary: $69875.00

--- Updated Employee Details ---

==================================================
EMPLOYEE DETAILS
==================================================
Employee ID: E001
Name: John Smith
Position: Software Engineer
Hire Date: 2020-03-15
Employment Duration: 5 years, 6 months
--------------------------------------------------
Annual Salary: $82,500.00
Monthly Salary: $6,875.00
==================================================


==================================================
EMPLOYEE DETAILS
```

```
==================================================
Employee ID: E003
Name: Mike Brown
Position: Data Analyst
Hire Date: 2021-01-10
Employment Duration: 4 years, 9 months
--------------------------------------------------
Annual Salary: $69,875.00
Monthly Salary: $5,822.92
==================================================


--- Finding Employee ---
Employee found: Sarah Johnson

==================================================
EMPLOYEE DETAILS
==================================================
Employee ID: E002
Name: Sarah Johnson
Position: Project Manager
Hire Date: 2019-06-20
Employment Duration: 6 years, 3 months
--------------------------------------------------
Annual Salary: $85,000.00
Monthly Salary: $7,083.33
==================================================


==================================================
PAYROLL SUMMARY
==================================================
Total Employees: 4
Total Annual Payroll: $307,375.00
Total Monthly Payroll: $25,614.58
==================================================


--- Specific Calculations ---
Sarah Johnson's Monthly Salary: $7,083.33
Sarah Johnson's Annual Salary: $85,000.00
Sarah Johnson's Employment Duration: 6 years, 3 months
```

QUESTION NO 7:

Create a `Product` class with attributes for product ID, name, price, and stock quantity. Design a `ShoppingCart` class that can add products, remove products, calculate total cost, and handle checkout process while updating inventory.

CODE

```python
class Product:
    """
    Represents a single product with its details.
    """
    def __init__(self, product_id, name, price, stock_quantity):
        self.product_id = product_id
```

```
        self.name = name
        self.price = price
        self.stock_quantity = stock_quantity

    def update_stock(self, quantity):
        """
        Updates the stock quantity. Decreases stock if the quantity is positive.
        """
        if self.stock_quantity >= quantity:
            self.stock_quantity -= quantity
            return True
        return False

    def __str__(self):
        """
        String representation of the Product object.
        """
        return f"ID: {self.product_id}, Name: {self.name}, Price:
${self.price:.2f}, Stock: {self.stock_quantity}"


class ShoppingCart:
    """
    Manages the shopping cart operations for a user like Hamna.
    """
    def __init__(self, user_name):
        self.user_name = user_name
        # The cart will store product objects and their desired quantity
        self.items = {}
        print(f"Shopping cart created for {self.user_name}.")

    def add_product(self, product, quantity=1):
        """
        Adds a product to the cart if it's in stock.
        """
        if product.stock_quantity >= quantity:
            if product.product_id in self.items:
                # If product is already in cart, just increase the quantity
                self.items[product.product_id]['quantity'] += quantity
            else:
                # Add the new product to the cart
                self.items[product.product_id] = {'product': product, 'quantity':
quantity}
            print(f"Added {quantity} x {product.name} to {self.user_name}'s cart.")
        else:
            print(f"Sorry, {product.name} is out of stock for the requested
quantity.")

    def remove_product(self, product):
        """
        Removes a product from the cart.
        """
        if product.product_id in self.items:
            removed_item = self.items.pop(product.product_id)
            print(f"Removed {removed_item['product'].name} from {self.user_name}'s
cart.")
        else:
            print(f"{product.name} is not in the cart.")

    def calculate_total_cost(self):
        """
        Calculates the total cost of all items in the cart.
        """
        total_cost = 0
        for item in self.items.values():
            total_cost += item['product'].price * item['quantity']
        return total_cost

    def display_cart(self):
        """
        Displays all the items currently in the shopping cart.
        """
        print("\n" + "="*30)
        print(f"Hamna's Shopping Cart Contents:")
        if not self.items:
```

```
            print("The cart is empty.")
        else:
            for item in self.items.values():
                product = item['product']
                quantity = item['quantity']
                print(f"- {product.name} (x{quantity}) - ${product.price *
quantity:.2f}")
        total = self.calculate_total_cost()
        print(f"Subtotal: ${total:.2f}")
        print("="*30 + "\n")


    def checkout(self):
        """
        Processes the checkout, updates inventory, and clears the cart.
        """
        total = self.calculate_total_cost()
        if total == 0:
            print("Your cart is empty. Nothing to checkout.")
            return

        print(f"Processing checkout for {self.user_name}...")
        print(f"Total amount to be paid: ${total:.2f}")

        # Update inventory for each product
        for item_id, item_details in self.items.items():
            product = item_details['product']
            quantity = item_details['quantity']
            product.update_stock(quantity)

        print("Payment successful! Inventory has been updated.")
        # Clear the cart after checkout
        self.items = {}
        print(f"{self.user_name}'s cart has been cleared. Thank you for shopping!")


# --- Main program execution ---
if __name__ == "__main__":
    # 1. Create some product instances
    product1 = Product(101, "AI Textbook", 55.00, 20)
    product2 = Product(102, "Graphics Card", 450.00, 10)
    product3 = Product(103, "Mechanical Keyboard", 120.50, 15)

    print("--- Initial Product Inventory ---")
    print(product1)
    print(product2)
    print(product3)
    print("\n" + "#"*40 + "\n")


    # 2. Create a shopping cart for Hamna
    hamna_cart = ShoppingCart("Hamna")

    # 3. Add products to the cart
    hamna_cart.add_product(product1, 2)  # Add 2 AI Textbooks
    hamna_cart.add_product(product2, 1)  # Add 1 Graphics Card
    hamna_cart.display_cart()

    # 4. Remove a product from the cart
    hamna_cart.remove_product(product1)
    hamna_cart.display_cart()

    # 5. Add another product
    hamna_cart.add_product(product3, 1)
    hamna_cart.display_cart()

    # 6. Proceed to checkout
    hamna_cart.checkout()
    hamna_cart.display_cart()

    print("\n" + "#"*40 + "\n")
    print("--- Final Product Inventory After Hamna's Purchase ---")
    print(product1)
    print(product2)
```

```
    print(product3)
```

OUTPUT

```
--- Initial Product Inventory --- ID: 101, Name: AI Textbook, Price: $55.00, Stock:
20 ID: 102, Name: Graphics Card, Price: $450.00, Stock: 10 ID: 103, Name: Mechanical
Keyboard, Price: $120.50, Stock: 15
########################################
Shopping cart created for Hamna. Added 2 x AI Textbook to Hamna's cart. Added 1 x
Graphics Card to Hamna's cart.
============================== Hamna's Shopping Cart Contents:
    • AI Textbook (x2) - $110.00
    • Graphics Card (x1) - $450.00 Subtotal: $560.00 ==============================
Removed AI Textbook from Hamna's cart.
============================== Hamna's Shopping Cart Contents:
    • Graphics Card (x1) - $450.00 Subtotal: $450.00 ==============================
Added 1 x Mechanical Keyboard to Hamna's cart.
============================== Hamna's Shopping Cart Contents:
    • Graphics Card (x1) - $450.00
    • Mechanical Keyboard (x1) - $120.50 Subtotal: $570.50
      ==============================
Processing checkout for Hamna... Total amount to be paid: $570.50 Payment successful!
Inventory has been updated. Hamna's cart has been cleared. Thank you for shopping!
============================== Hamna's Shopping Cart Contents: The cart is empty.
Subtotal: $0.00
########################################
--- Final Product Inventory After Hamna's Purchase --- ID: 101, Name: AI Textbook,
Price: $55.00, Stock: 20 ID: 102, Name: Graphics Card, Price: $450.00, Stock: 9 ID:
103, Name: Mechanical Keyboard, Price: $120.50, Stock: 14
```

QUESTION NO 8:

Design a `Patient` class to store patient information (ID, name, age, medical history) and an `Appointment` class to manage doctor appointments. Include functionality to schedule, cancel, and view appointments.

CODE

```
class Patient:
    """
    Represents a patient with their personal and medical information.
    Authored by Hamna for the AI Lab.
    """
    def __init__(self, patient_id, name, age, medical_history):
        self.patient_id = patient_id
        self.name = name
        self.age = age
        self.medical_history = medical_history

    def __str__(self):
        """
        Provides a string representation of the Patient object.
        """
        return (f"Patient ID: {self.patient_id} | Name: {self.name} | "
                f"Age: {self.age} | History: {self.medical_history}")

class AppointmentScheduler:
    """
    Manages scheduling, viewing, and canceling appointments at Hamna's clinic.
```

```
    """
    def __init__(self, clinic_name):
        self.clinic_name = clinic_name
        self.appointments = {}  # Using a dictionary for easy access by ID
        self.next_appointment_id = 1
        print(f"Welcome to the {self.clinic_name} Appointment Scheduler!")

    def schedule_appointment(self, patient, doctor_name, date_time):
        """
        Schedules a new appointment for a given patient.
        """
        appointment_id = self.next_appointment_id
        self.appointments[appointment_id] = {
            'patient': patient,
            'doctor': doctor_name,
            'datetime': date_time,
            'status': 'Scheduled'
        }
        self.next_appointment_id += 1
        print(f"\nSUCCESS: Appointment #{appointment_id} for {patient.name} with "
              f"Dr. {doctor_name} on {date_time} has been scheduled.")
        return appointment_id

    def cancel_appointment(self, appointment_id):
        """
        Cancels an existing appointment using its ID.
        """
        if appointment_id in self.appointments:
            patient_name = self.appointments[appointment_id]['patient'].name
            # Instead of deleting, we can just update the status
            # self.appointments[appointment_id]['status'] = 'Canceled'
            # Or we can remove it entirely
            del self.appointments[appointment_id]
            print(f"\nSUCCESS: Appointment #{appointment_id} for {patient_name} has
been canceled.")
        else:
            print(f"\nERROR: Appointment #{appointment_id} not found.")

    def view_appointments(self, patient_filter=None):
        """
        Displays all appointments. Can be filtered by a specific patient.
        """
        print("\n" + "="*40)
        if patient_filter:
            print(f"Viewing Appointments for Patient: {patient_filter.name}")
        else:
            print(f"Viewing All Appointments at {self.clinic_name}")
        print("="*40)

        if not self.appointments:
            print("No appointments scheduled.")
            return

        found_appointments = False
        for app_id, details in self.appointments.items():
            if patient_filter is None or details['patient'].patient_id ==
patient_filter.patient_id:
                found_appointments = True
                print(f"  Appointment ID: {app_id}")
                print(f"    Patient: {details['patient'].name} (ID:
{details['patient'].patient_id})")
                print(f"    Doctor: Dr. {details['doctor']}")
                print(f"    Date & Time: {details['datetime']}")
                print(f"    Status: {details['status']}")
                print("-" * 20)

        if not found_appointments:
            if patient_filter:
                print(f"No appointments found for {patient_filter.name}.")
            else:
                print("No appointments in the system.")

        print("="*40 + "\n")
```

```
# --- Main program execution by Hamna ---
if __name__ == "__main__":
    # 1. Create patient instances
    patient_hamna = Patient(1, "Hamna", 21, "None")
    patient_ali = Patient(2, "Ali Khan", 35, "Allergic to penicillin")
    patient_sana = Patient(3, "Sana Ahmed", 42, "High blood pressure")

    print("--- Patients Registered ---")
    print(patient_hamna)
    print(patient_ali)
    print(patient_sana)
    print("\n")

    # 2. Initialize the appointment scheduler
    scheduler = AppointmentScheduler("Hamna's AI Clinic")

    # 3. Schedule some appointments
    app1_id = scheduler.schedule_appointment(patient_hamna, "Iqbal", "2025-10-15
10:00 AM")
    app2_id = scheduler.schedule_appointment(patient_ali, "Fatima", "2025-10-15
11:30 AM")
    app3_id = scheduler.schedule_appointment(patient_sana, "Iqbal", "2025-10-16
09:00 AM")

    # 4. View all current appointments
    scheduler.view_appointments()

    # 5. Cancel an appointment
    scheduler.cancel_appointment(app2_id) # Cancel Ali's appointment

    # 6. View all appointments again to see the change
    scheduler.view_appointments()

    # 7. View appointments for a specific patient
    scheduler.view_appointments(patient_hamna)
```

QUESTION NO 9:

Create a `User` class for a social media platform with attributes for username, email, friends list, and posts. Implement methods to add friends, create posts, and display user timeline. Include privacy settings for posts.

CODE

```
class User:
    def __init__(self, username, email):
        # Basic information
        self.username = username
        self.email = email

        # Empty lists for friends and posts
        self.friends = []
        self.posts = []

    def add_friend(self, friend_name):
        """Add a friend to the friend list"""
        if friend_name not in self.friends:
            self.friends.append(friend_name)
            print(f"{friend_name} added as a friend!")
        else:
            print(f"{friend_name} is already your friend.")

    def create_post(self, content, privacy):
        """
        Create a new post with content and privacy setting.
        privacy = 'public' or 'private'
        """
```

```
        post = {"content": content, "privacy": privacy}
        self.posts.append(post)
        print("Post created successfully!")

    def show_timeline(self):
        """Display all public posts"""
        print(f"\nTimeline of {self.username}:")
        if not self.posts:
            print("No posts yet!")
        else:
            for i, post in enumerate(self.posts, start=1):
                if post["privacy"] == "public":
                    print(f"{i}. {post['content']} (Public)")
                else:
                    print(f"{i}. {post['content']} (Private)")

# ----------------------------
# Example usage by Hamna
# ----------------------------
# Creating user
hamna = User("Hamna", "hamna@example.com")

# Adding friends
hamna.add_friend("Aisha")
hamna.add_friend("Fatima")

# Creating posts
hamna.create_post("My first post!", "public")
hamna.create_post("Having a lazy Sunday", "private")
hamna.create_post("Python practice is fun!", "public")

# Displaying user timeline
hamna.show_timeline()
```

OUTPUT

```
Aisha added as a friend!
Fatima added as a friend!
✅ Post created successfully!
✅ Post created successfully!
✅ Post created successfully!

Timeline of Hamna:
1. My first post! (Public)
2. Having a lazy Sunday (Private)
3. Python practice is fun! (Public)
```

QUESTION NO 10:

Design a `Room` class with room number, type (single/double/suite), price, and availability. Create a `Booking` class to handle room reservations, check-in/check-out dates, and calculate total stay cost.

CODE:

```
class Room:
    def __init__(self, room_no, room_type, price, available=True):
        self.room_no = room_no
        self.room_type = room_type
        self.price = price
        self.available = available

class Booking:
    def __init__(self, room, check_in, check_out):
        self.room = room
        self.check_in = check_in
        self.check_out = check_out
```

```
    def calculate_cost(self, days):
        return self.room.price * days

# Example usage
room1 = Room(101, "Single", 5000)
booking1 = Booking(room1, "2025-10-20", "2025-10-23")

print("Room No:", booking1.room.room_no)
print("Total Cost:", booking1.calculate_cost(3))
```

OUTPUT

```
Room No: 101
Total Cost: 15000
```

QUESTION NO 11:

Create `Course` classes (with code, name, credits, capacity) and a `Student` class that can register for courses. Implement functionality to check for schedule conflicts and ensure students don't exceed credit limits.

CODE

```
class Course:
    def __init__(self, code, name, credits, capacity):
        self.code = code
        self.name = name
        self.credits = credits
        self.capacity = capacity

class Student:
    def __init__(self, name):
        self.name = name
        self.courses = []
        self.total_credits = 0

    def register_course(self, course):
        if self.total_credits + course.credits > 18:
            print("Credit limit exceeded!")
        elif course.capacity <= 0:
            print("Course is full!")
        else:
            self.courses.append(course)
            self.total_credits += course.credits
            course.capacity -= 1
            print(f"{self.name} registered for {course.name}")

# Example
c1 = Course("CS101", "Python", 3, 2)
student1 = Student("Hamna")

student1.register_course(c1)
```

OUTPUT

```
Hamna registered for Python
```

QUESTION NO 12

Design a `Product` class for items in inventory (ID, name, quantity, price, supplier) and an `Inventory` class to track stock levels, add new products, update quantities, and generate low-stock alerts.

CODE

```
class Product:
    def __init__(self, pid, name, quantity, price, supplier):
        self.pid = pid
        self.name = name
        self.quantity = quantity
        self.price = price
        self.supplier = supplier

class Inventory:
    def __init__(self):
        self.products = []

    def add_product(self, product):
        self.products.append(product)

    def update_quantity(self, pid, qty):
        for p in self.products:
            if p.pid == pid:
                p.quantity += qty

    def low_stock_alert(self):
        for p in self.products:
            if p.quantity < 5:
                print(f"Low stock: {p.name}")

# Example
inv = Inventory()
p1 = Product(1, "Keyboard", 3, 800, "Hamna Traders")
inv.add_product(p1)
inv.low_stock_alert()
```

OUTPUT

```
Low stock: Keyboard
```

QUESTION NO 13

Create `Movie` classes (title, genre, duration, showtimes) and a `Theater` class with seating arrangement. Implement a booking system that reserves seats and calculates ticket prices with different categories (standard, premium).

CODE

```
class Movie:
    def __init__(self, title, genre, duration, showtimes):
        self.title = title
        self.genre = genre
        self.duration = duration
```

```
        self.showtimes = showtimes

class Theater:
    def __init__(self):
        self.seats = {"Standard": 300, "Premium": 600}
        self.booked = 0

    def book_ticket(self, category):
        if category in self.seats:
            price = self.seats[category]
            print(f"Ticket booked! Price: Rs.{price}")
            self.booked += 1
        else:
            print("Invalid category")

# Example
movie = Movie("Frozen", "Animation", 120, ["6PM", "9PM"])
theater = Theater()
theater.book_ticket("Premium")
```

OUTPUT

```
Ticket booked! Price: Rs.600
```

QUESTION NO 14

Design a `Member` class with personal details, membership type (basic/premium), and attendance records. Create a `Trainer` class and a system to schedule training sessions between members and trainers.

CODE

```
class Member:
    def __init__(self, name, m_type):
        self.name = name
        self.m_type = m_type
        self.attendance = 0

class Trainer:
    def __init__(self, name):
        self.name = name

    def schedule_session(self, member):
        print(f"Session scheduled for {member.name} with trainer {self.name}")

# Example
m1 = Member("Hamna", "Premium")
t1 = Trainer("Hania")
t1.schedule_session(m1)
```

OUTPUT

```
Session scheduled for Hamna with trainer Hania
```

QUESTION NO 15

Extend the bank account system to include `SavingsAccount` and `CheckingAccount` classes with different interest rates and withdrawal rules. Implement fund transfer between accounts.

CODE

```
class BankAccount:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance

class SavingsAccount(BankAccount):
    def __init__(self, name, balance, interest_rate):
        super().__init__(name, balance)
        self.interest_rate = interest_rate

class CheckingAccount(BankAccount):
    def withdraw(self, amount):
        if self.balance >= amount:
            self.balance -= amount
            print("Withdrawal successful")
        else:
            print("Insufficient funds")

# Example
s1 = SavingsAccount("Hamna", 10000, 0.05)
c1 = CheckingAccount("Imran", 5000)
c1.withdraw(2000)
```

OUTPUT

```
Withdrawal successful
```

QUESTION NO 16

Create a `Pizza` class with size, crust type, and toppings. Design an ordering system that calculates prices based on selections and allows custom pizza creation with various topping combinations.

CODE

```
class Pizza:
    def __init__(self, size, crust, toppings):
        self.size = size
        self.crust = crust
        self.toppings = toppings

    def calculate_price(self):
        base_price = {"small": 500, "medium": 800, "large": 1200}
        topping_price = 100 * len(self.toppings)
        return base_price[self.size] + topping_price

# Example
pizza1 = Pizza("medium", "Cheese Burst", ["Olives", "Mushrooms"])
print("Total Price:", pizza1.calculate_price())
```

OUTPUT

```
Total Price: 1000
```

QUESTION NO 17

Design classes for `Car` (model, year, color, price) and `CarManufacturer` that can produce different car models with various features. Include quality control checks and production tracking.

CODE

```
class Car:
    def __init__(self, model, year, color, price):
        self.model = model
        self.year = year
        self.color = color
        self.price = price

class CarManufacturer:
    def __init__(self):
        self.cars = []

    def produce_car(self, car):
        self.cars.append(car)
        print(f"Produced: {car.model}")

# Example
car1 = Car("Civic", 2025, "White", 4500000)
factory = CarManufacturer()
factory.produce_car(car1)
```

OUTPUT

```
Produced: Civic
```

QUESTION NO 18

Create `Teacher` and `Student` classes, and a `Classroom` class that manages assignments, grades, and attendance. Implement functionality to track student performance and generate class reports.

CODE

```
class Student:
    def __init__(self, name):
        self.name = name
        self.grades = []

class Teacher:
    def __init__(self, name):
        self.name = name

class Classroom:
    def __init__(self):
        self.students = []

    def add_student(self, student):
        self.students.append(student)

    def record_grade(self, student, grade):
        student.grades.append(grade)

# Example
s1 = Student("Hamna")
t1 = Teacher("Sir Imran")
room = Classroom()
room.add_student(s1)
room.record_grade(s1, 95)
print(f"{s1.name}'s Grades:", s1.grades)
```

OUTPUT

```
Hamna's Grades: [95]
```

QUESTION NO 19

Design `Flight` classes (flight number, destination, departure time, capacity) and a `Passenger` class. Create a booking system that reserves seats and manages passenger manifests.

CODE

```
class Flight:
    def __init__(self, number, destination, capacity):
        self.number = number
        self.destination = destination
        self.capacity = capacity
        self.passengers = []

class Passenger:
    def __init__(self, name):
        self.name = name

    def book_seat(self, flight):
        if len(flight.passengers) < flight.capacity:
            flight.passengers.append(self.name)
            print(f"{self.name} booked on flight {flight.number}")
        else:
            print("Flight full")

# Example
f1 = Flight("PK101", "Karachi", 2)
p1 = Passenger("Hamna")
p1.book_seat(f1)
```

OUTPUT

```
Hamna booked on flight PK101
```

QUESTION NO 20

Create classes for `SmartDevice` (base class) and specific devices like `SmartLight`, `Thermostat`, and `SecurityCamera`. Implement a `SmartHome` class that can control all devices and create automation routines

CODE

```
class SmartDevice:
    def __init__(self, name):
        self.name = name
        self.status = "off"

    def turn_on(self):
        self.status = "on"
        print(f"{self.name} turned ON")

class SmartLight(SmartDevice):
    pass

class Thermostat(SmartDevice):
```

```
    def set_temp(self, temp):
        print(f"Temperature set to {temp}°C")

class SecurityCamera(SmartDevice):
    def record(self):
        print(f"{self.name} is recording...")

class SmartHome:
    def __init__(self):
        self.devices = []

    def add_device(self, device):
        self.devices.append(device)

# Example
home = SmartHome()
light = SmartLight("Bedroom Light")
cam = SecurityCamera("Front Camera")
home.add_device(light)
home.add_device(cam)
light.turn_on()
cam.record()
```

OUTPUT

```
Bedroom Light turned ON
Front Camera is recording...
```